

Stage 1 Lab - Human vs AI: Building Your First SmartCare Prototype

Learning objectives

- Create and run a simple Python file with basic input,output and processing statements
- Use lists, dictionaries and functions to enhance the Python file
- Build a small SmartCare appointment prototype.
- Use AI as a tutor rather than a replacement.
- Compare human-written and AI-generated code.
- Verify AI-generated code through execution and test inputs.
- Document a short AI-use reflection.

Files to create and commit in GitHub

```
stage01/  
  smartcare_v01.py  
  comparison.md  
  reflection.md  
  ai_usage.md
```

Part A - Understand the Problem: AI OFF

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored?

Patient name, practitioner name and appointment time

What functions might be useful?

Functions that can book an appointment and display appointments respectively.

What could go wrong?

- Values could be missing (names or appointment times)
- Invalid values are entered
- Double bookings for the same practitioner

What requirements are unclear?

- Who manages practitioner availability
- Who can make/edit appointment bookings or cancellations
- Whether a practitioner can have more than one appointment booked at the same time
- Whether appointments remain in the list even if it is cancelled

Part B - Build a Human-Written Prototype: AI OFF

#task 1

Create and run a simple Python file with basic input,output statements

```
print("Welcome to SmartCare: Community Clinic Appointment Booking System!")
```

First Appointment

```
patient1_name = 'Alice Smith'
```

```
practitioner1_name = 'Dr. John Doe'
```

```
appointment1_time = '2024-07-20 10:00 AM'
```

```
print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time: {appointment1_time}")
```

Second Appointment

```
patient2_name = 'Bob Johnson'
```

```
practitioner2_name = 'Dr. Jane Roe'
```

```
appointment2_time = '2024-07-20 11:30 AM'
```

```
print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time: {appointment2_time}")
```

.....

#task1enhanced

Use lists, dictionaries and functions to enhance the Python file

```
appointments = []
```

```
def book_appointment(patient_name, practitioner_name, appointment_time):
```

```
    if not patient_name:
```

```
        raise ValueError("Patient name cannot be empty")
```

```
    appointment = {
```

```
        "patient": patient_name,
```

```
        "practitioner": practitioner_name,
```

```
        "time": appointment_time
```

```
    }
```

```
    appointments.append(appointment)
```

```
def display_appointments():
```

```
    if not appointments:
```

```
        print("No appointments recorded.")
```

```
        return
```

```
    for appointment in appointments:
```

```
        print(f"Patient: {appointment['patient']} | Practitioner:
```

```
{appointment['practitioner']} | Time: {appointment['time']}")
```

```
print("Welcome to SmartCare: The Clinical Appointment Booking System!")
book_appointment('Alice Smith', 'Dr. John Doe', '2024-07-20 10:00 AM')
book_appointment('Bob Johnson', 'Dr. Jane Roe', '2024-07-20 11:30 AM')
display_appointments()
```

^ Now, run both programs , and identify at least five limitations.

1. Double bookings can occur since the program does not check for existing appointments
2. Practitioner name is not validated which causes missing information
3. Appointment time is not validated which causes missing information
4. Appointment time is defined as plain text, therefore any input will be defined as valid
5. Data / appointment information disappears when closing the program

Part C - Use AI as Tutor: AI ON (Use only UC approved GenAI Tool such as Microsoft CoPilot)

Suggested prompt structure:

Act as a Python tutor.

I am learning introductory software technology.

Here is a small appointment-booking function.

1. Explain what the code does.
2. Identify three limitations.
3. Suggest improvements.
4. Do not rewrite the whole application.
5. Ask me two questions to test my understanding.

Summarised AI Response:

1. Explain what the code does
 - a. Creates an empty list
 - b. Defines a function
 - c. Check that the patient name is not empty
 - d. Creates a dictionary
 - e. Adds the appointment to the list
2. Limitations
 - a. Only the patient name is validated
 - b. No validation of appointment time format
 - c. Duplicate appointments are possible
3. Suggested improvements
 - a. Validate all inputs
 - b. Validate the time format
 - c. Prevent duplicate bookings
5. Questions testing understanding
 - a. What happens when this code runs? `book_appointment("", "Dr Smith", "10:00")` Does an appointment get added to the list? What exception is raised?

- b. After running: `book_appointment("Alice", "Dr Smith", "10:00")`, What is stored in the appointments list? Try to describe the contents of the list and the dictionary it contains.

Part D - Generate an Alternative: AI ON

Ask AI to create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Explicitly prohibit a database or GUI.

```
appointments = []

def book_appointment(patient_name, practitioner_name, appointment_time):
    # Check for empty values
    if not patient_name:
        raise ValueError("Patient name cannot be empty")
    if not practitioner_name:
        raise ValueError("Practitioner name cannot be empty")
    if not appointment_time:
        raise ValueError("Appointment time cannot be empty")

    # Check for duplicate appointment
    for appointment in appointments:
        if (appointment["practitioner"] == practitioner_name
            and appointment["time"] == appointment_time):
            raise ValueError("This practitioner is already booked at that time")

    # Create and store the appointment
    appointment = {
        "patient": patient_name,
        "practitioner": practitioner_name,
        "time": appointment_time
    }

    appointments.append(appointment)
```

```
print("Appointment booked successfully.")
```

```
# Example usage
```

```
book_appointment("Alice Brown", "Dr Smith", "10:00 AM")
```

```
book_appointment("John Lee", "Dr Jones", "2:30 PM")
```

```
# Display all appointments
```

```
print("\nAppointments:")
```

```
print(appointments)
```

Part E - Compare Human and AI Versions

Question	Human version	AI version
Easy to understand?	Yes	Yes
Runs successfully?	Yes	Yes
Uses only required features?	Yes	Yes
Adds assumptions?	Assumes storing time as a string and without a validated time format is sufficient	Assumes there will always be an appointment when running the program, therefore there is no if statement that addresses scenarios where there are no appointments in the list
Handles errors?	Only validates patient name	Based on improvements it suggested, it does not address validations for appointment time format
Could I explain it?	Yes	Yes

Part F - Verify Behaviour

- Normal appointment
- Blank patient name
- Two appointments for the same practitioner/time
- Strange input such as patient_name=None or appointment_time=None

Verify	Human	AI
Normal appointment	Prints formatted appointment details via display_appointments()	Prints appointment details. If there is more than one, than it prints all appointments in a single line
Blank patient name	Runs ValueError	Runs ValueError
Duplicate appointment	Ignores	Runs ValueError, however does not display any of the other appointments as a result
Strange input	Validates patient_name but does not validate practitioner_name or appointment_time	Runs ValueError for any missing values for patient_name, practitioner_name and appointment_time

Part G - Improve One Thing

Choose exactly one controlled improvement, for example: if not patient_name: raise ValueError("Patient name cannot be empty")

if not practitioner_name: raise ValueError("Practitioner name cannot be empty")

Part H - Reflection (150-250 words)

What did you build before using AI?

What did AI help you understand?

Did AI make assumptions?

How did you verify the AI output?

What engineering work remained for you?

Before using AI, by utilising inputs, outputs, functions, dictionaries and lists aligning with current Python programming skills, a simple appointment management prototype is made. To elaborate, it used a function such as `book_appointment()` to store values for `patient_name`, `practitioner_name` and `appointment_time`. To prevent `ValueErrors`, *raise* validated the `patient_name` to ensure there were no empty fields in the input. In addition, the `display_appointments()` formatted the inputted appointment details into a list of all appointments.

After asking AI, it helped me understand how sections of the code worked using the appropriate terminology relevant to Python programming. AI helped me understand how to validate duplicate bookings by using an `if`-statement to capture whether there was an existing appointment by referring back to the `practitioner_name` and `appointment_time`.

Based on the code AI provided, it assumed there will always be an appointment already in the list when running the program, therefore there is no return statement to end the function. AI did not assume any implementation for GUI and databases.

To verify the AI output, I ran the code in PyCharm and ran tests regarding normal appointments, blank patient names, duplicate appointments, and strange inputs. As a result, I was able to verify that the output from AI worked, however it had issues with visually displaying appointments in a single line.

Engineering work that remained for me was being able to understand how the code works, evaluating an AI version of the code, verifying both codes and ensuring it aligned with the clinic's software requirements.

Submission checklist [GitHub Commit]

- Python file runs.
- Comparison table completed.
- Normal and unusual inputs tested.
- AI assistance documented.
- Reflection completed.
- I can explain my code.